









when (x) {  }

when (x) {



x.balanced += 10;





An Axiomatic Concurrency Model

4

9

S



R



C



Each behavior has three life cycle events: Spawn, Run and Complete

Build and maintain excellent



Exeutions are the events of the program and their order

S



po



po

R

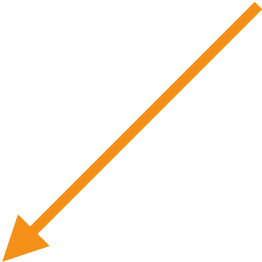






po

poor is the program order



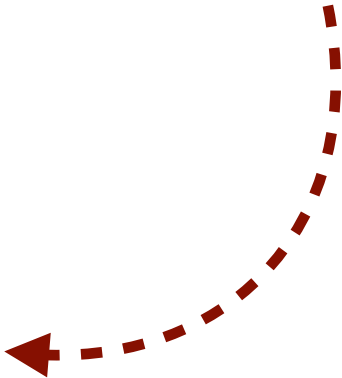
co is the crown order

COX





ris the derived run order



h

b

h is the derivative happens before

**If behaviours share a cown and have a
path between their spawns then
complete happens-before run**





when (x) {

A light blue circle with a white letter 'B' inside, centered within the code block.

}

when (x) {



x.balance += 10;

Exeutions are valid if ayclic









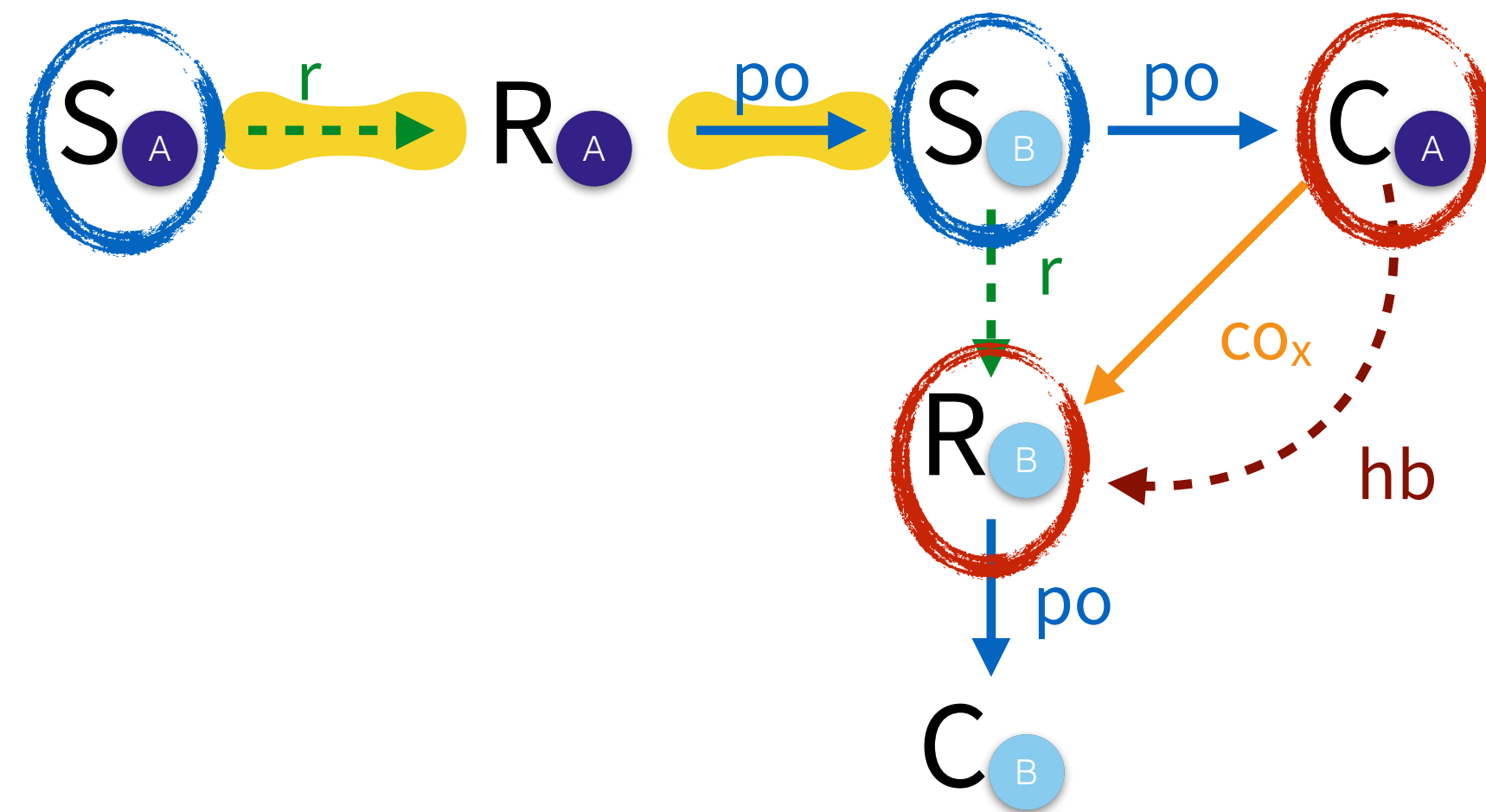
An Axiomatic Concurrency Model

```
when(x) { A
  x.balance += 10;
  when(x) { B }
}
```

Build around candidate executions

Executions are the events of the program and their order

Each behaviour has three lifecycle events: Spawn, Run and Complete



po is the program order

co is the cown order

r is the derived run order

hb is the derived happens before order

If **behaviours share a cown** and have a **path between their spawns** then **complete happens-before run**

Executions are valid if acyclic

Validating Causality

```
def update(acc: cown[Account], amount: int) {  
  when(acc) { A  
    acc.balance += amount  
    val id = acc.id()  
    when(log) { B log.record(id, amount) }  
}
```

update(src, +100)

update(src, -100)